actually induces compositional representations. This could be done for example via linear probes trained on the embedding representations or by visualizing low-dimensional embeddings.

Potential Violation of Code of Ethics: No.

C.1.2. AI Scientist Team Code Review

Inspecting the dataset generation process

The data-generating function, which uses a single-digit expression as shown in fig. 4, generates at most $81 * k$ possible combinations, where k is the number of operators. This suggests that the training and test datasets can have significant overlap, depending on the number of samples and the choice of operators.

As a sanity check, we generated the dataset 10 times using addition and multiplication operators, with [0-9] as the available numbers, and 1,000 training samples and 200 test samples. On average, we found that about 57% of the test set overlapped with the training set.

```
# Generate synthetic data with varying operator complexity
def generate_expression_data(n_samples, operator_set):
    numbers = list(range(1, 10))
    expressions = []
    results = []

    for _ in range(n_samples):
        num1, num2 = np.random.choice(numbers, 2)
        op = np.random.choice(operator_set)
        expr = f"{num1}{op}{num2}"

        # Handle division by zero
        if op == "/" and num2 == 0:
            num2 = np.random.choice([n for n in numbers if n != 0])
            expr = f"{num1}{op}{num2}"

        result = eval(expr)
        expressions.append(expr)
        results.append(result)

    return expressions, results


# Create vocabulary including all possible operators
vocab = list("0123456789+-*/")
char2idx = {c: i for i, c in enumerate(vocab)}
idx2char = {i: c for c, i in char2idx.items()}
vocab_size = len(vocab)
```

Figure 4 | Example of the data generating function used in the experiments.

Model architecture, loss function, and evaluation function

While the model architecture is simple, its implementation appears to be correct, as shown in fig. 5.

In the training loop, presented in fig. 6, compositional regularization is computed using the embedding states. Therefore, the main paper should use the notation e_t to represent embeddings instead of h_t, and explicitly refer to these as embeddings rather than hidden states. Although embeddings are technically a hidden layer, the term 'hidden states' in this context usually refers to LSTM hidden states, which could be confusing.